E2E Setup Log

This project uses Playwright as the primary toolkit for E2E testings. The reasons why we have chosen it are as follows:

- Playwright is made for web app development testing. It is appropriate for this project.
- As it is an open-source project, it's free.
- Unlimited usage. No monthly limit on how many tests you can do.
- Cross compatibility with different browsers, platforms and coding languages.

Below is a guide that provides instructions on setting up the E2E testing environment.

Getting Started

- Run all commands from within the `porterest` directory.

1. **Install Playwright:** You can also follow the official instructions at playwright.dev

```
npm init playwright@latest
```

2. **Update Playwright:**

```
npm install -D @playwright/test@latest
npx playwright install --with-deps
```

3. **Check your installed version:**

```
npx playwright --version
```

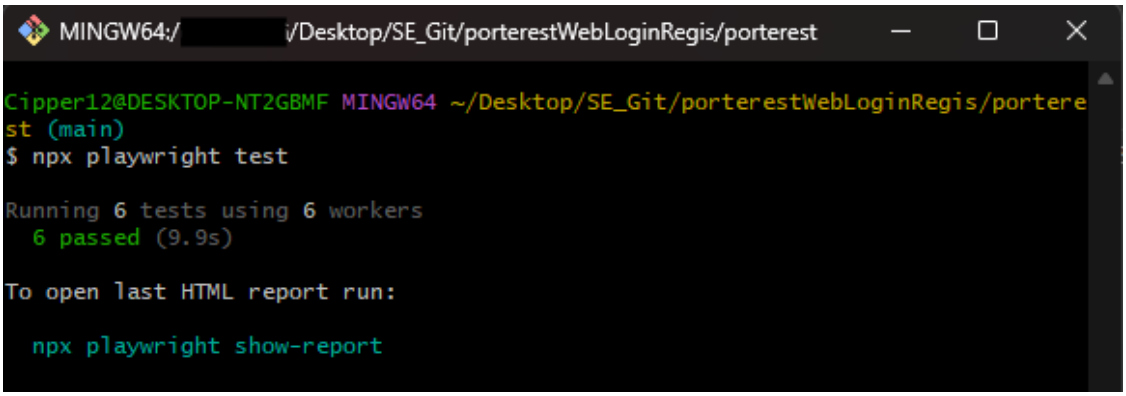
Running the Example Test

After Playwright has been installed with the latest update you can verify it by running the included example test. You can use default mode to display output and the result in the terminal or use the UI mode to do the same with live step view, time travel debugging and more.

Default Mode

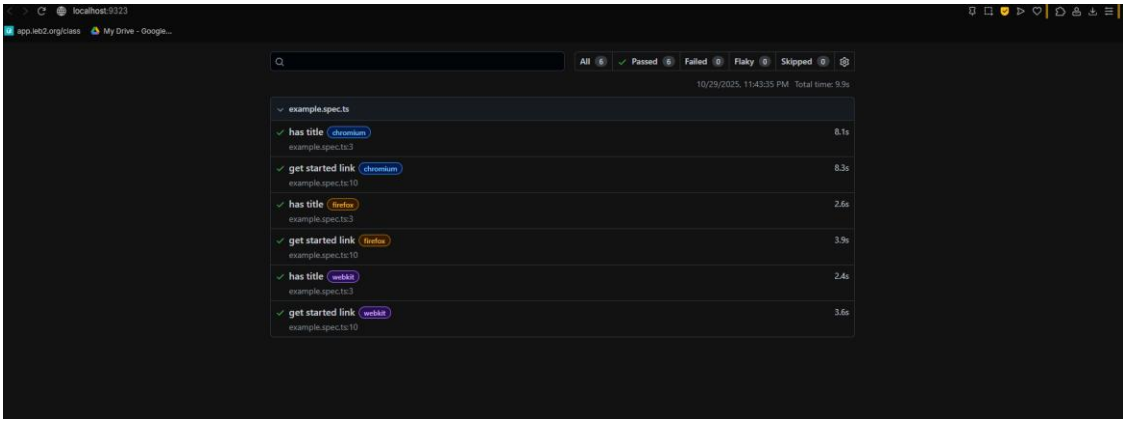
To run the example test:

```
npx playwright test
```



To check the test report:

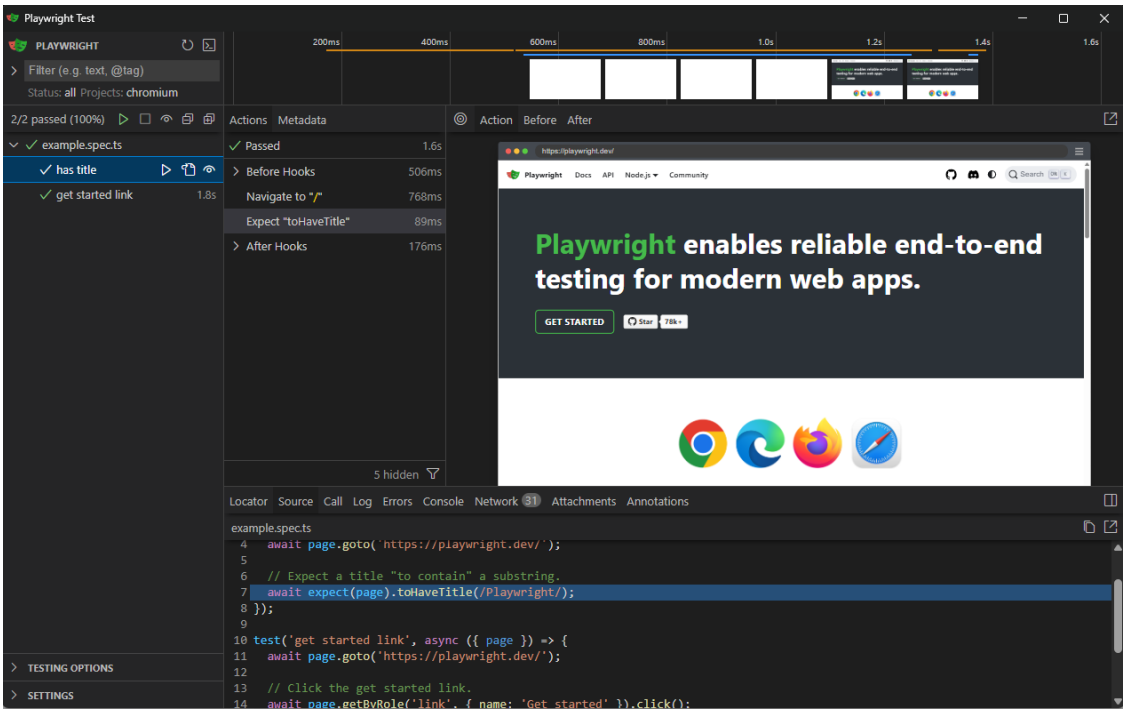
```
npx playwright show-report
```



UI Mode

To run the example test in UI mode:

```
npx playwright test --ui
```



E2E Test Plan

Scenario ID: E2E-001

Title: User login and dashboard access (Google OAuth)

Precondition: Google OAuth configured

Steps:

1. Open home page (/Login)
2. Click Sign in with Google
3. After callback navigate to /Dashboard
4. Verify username/email visible

Expected: Dashboard shows authenticated user info

Scenario ID: E2E-002

Title: User registration

Precondition: No account tied to the email used for registration.

Steps:

1. Open home page
2. Click Sign Up
3. Enter appropriate email and password
4. Click Register button
5. After successful account registration navigate to /FinishSignup
6. Redirect to /Dashboard
7. Verify username/email visible

Expected: Account is created. Information is stored in the database. Dashboard shows authenticated user info

Scenario ID: E2E-003

Title: User login

Precondition: Account must already be registered.

Steps:

1. Open home page
2. Enter appropriate email and password
3. Click Login button
4. After successful login navigate to /Dashboard
5. Verify username/email visible

Expected: Dashboard shows authenticated user info

Scenario ID: E2E-004

Title: Unauthenticated page access

Precondition: User isn't logged in and tries to access a page directly via URL

Steps:

1. Open a link to /Dashboard directly via clicking a link or entering it into the search engine
2. The system detected unauthorized access to the page
3. Redirect to home page

Expected: Unauthorized access get redirect to home page

Test Implementation

// tests/e2e/saveAuthStorage.js

```
import { chromium } from '@playwright/test';

(async () => {
  const browser = await chromium.launch({ headless: false });
  const context = await browser.newContext();
  const page = await context.newPage();

  await page.goto('http://localhost:5173');
  await page.getByText('Sign in with Google', { exact: false }).click();

  console.log('Complete the Google sign-in in the opened browser. After redirect back to app, press Enter here to save storage.');
  process.stdin.setRawMode(true);
  await new Promise(resolve => process.stdin.once('data', resolve));

  await context.storageState({ path: 'tests/e2e/authStorage.json' });
  console.log('Saved storage state to tests/e2e/authStorage.json');

  await browser.close();
  process.exit(0);
})();
```

// tests/e2e/auth.spec.js

```
import { test, expect } from '@playwright/test';

test.use({ storageState: 'tests/e2e/authStorage.json' });
test('user logs in and sees dashboard', async ({ page }) => {
  await page.goto('http://localhost:5173/dashboard');
  await expect(page.locator('h1')).toContainText('Dashboard');
});
```

Terminal:

```
Complete the Google sign-in in the opened browser. After redirect back to app, press Enter here to save storage.
Saved storage state to tests/e2e/authStorage.json
PS C:\Users\ASUS\Desktop\SE\porterestWebLoginRegis\porterest> npx playwright test tests/e2e/auth.spec.js --headed

Running 3 tests using 3 workers
  3 passed (8.9s)

To open last HTML report run:

  npx playwright show-report

PS C:\Users\ASUS\Desktop\SE\porterestWebLoginRegis\porterest>
PS C:\Users\ASUS\Desktop\SE\porterestWebLoginRegis\porterest>  npx playwright show-report

  Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```

playwright report:

All 3 | Passed 3 | Failed 0 | Flaky 0 | Skipped 0

10/30/2025, 8:04:45 AM Total time: 8.9s

e2e/auth.spec.js

user logs in and sees dashboard chromium e2e/auth.spec.js:5		1.2s
user logs in and sees dashboard firefox e2e/auth.spec.js:5		5.2s
user logs in and sees dashboard webkit e2e/auth.spec.js:5		2.3s

Unit & Integration Tests

Test run screenshot (error message is expected for failure case, all tests passed):

```
bun run test

 RUN  v4.0.5 D:/university/Year 3/Classes/CPE334 SOFTWARE ENGINEERING/Projects/porterest-repo/porterest

stderr | tests/frontend/AuthContext.test.jsx > AuthContext > should handle fetch errors gracefully
Session check failed: Error: Network error
    at D:/university/Year 3/Classes/CPE334 SOFTWARE ENGINEERING/Projects/porterest-repo/porterest/tests/frontend/AuthContext.test.jsx:73:40
    at file:///D:/university/Year%203/Classes/CPE334%20SOFTWARE%20ENGINEERING/Projects/porterest-repo/porterest/node_modules/@vitest/runner/dist/index.js:157:11
    at file:///D:/university/Year%203/Classes/CPE334%20SOFTWARE%20ENGINEERING/Projects/porterest-repo/porterest/node_modules/@vitest/runner/dist/index.js:753:26
    at file:///D:/university/Year%203/Classes/CPE334%20SOFTWARE%20ENGINEERING/Projects/porterest-repo/porterest/node_modules/@vitest/runner/dist/index.js:1636:20
    at new Promise (<anonymous>)
    at runWithTimeout (file:///D:/university/Year%203/Classes/CPE334%20SOFTWARE%20ENGINEERING/Projects/porterest-repo/porterest/node_modules/@vitest/runner/dist/index.js:1602:10)
    at runTest (file:///D:/university/Year%203/Classes/CPE334%20SOFTWARE%20ENGINEERING/Projects/porterest-repo/porterest/node_modules/@vitest/runner/dist/index.js:1309:12)
    at runSuite (file:///D:/university/Year%203/Classes/CPE334%20SOFTWARE%20ENGINEERING/Projects/porterest-repo/porterest/node_modules/@vitest/runner/dist/index.js:1468:8)
    at runSuite (file:///D:/university/Year%203/Classes/CPE334%20SOFTWARE%20ENGINEERING/Projects/porterest-repo/porterest/node_modules/@vitest/runner/dist/index.js:1468:8)
    at runFiles (file:///D:/university/Year%203/Classes/CPE334%20SOFTWARE%20ENGINEERING/Projects/porterest-repo/porterest/node_modules/@vitest/runner/dist/index.js:1526:3)

 ✓ tests/frontend/AuthContext.test.jsx (3 tests) 96ms
 ✓ tests/frontend/ProtectedRoute.test.jsx (3 tests) 67ms
 ✓ tests/frontend/FinishSignup.test.jsx (2 tests) 371ms

 Test Files  3 passed (3)
      Tests  8 passed (8)
   Start at  18:01:13
   Duration  3.07s (transform 386ms, setup 791ms, collect 1.69s, tests 534ms, environment 4.52s, prepare 79ms)

$ bun test --preload ./tests/backend/setup.js ./tests/backend/
bun test v1.3.0 (b0a6feca)

tests\backend\auth.test.js:
✓ Authentication Endpoints > GET /auth/google/url > should return a Google auth URL and set CSRF cookies [15.00ms]
✓ Authentication Endpoints > GET /auth/google/callback > should return 400 if state is invalid
Authenticated existing user: existing@test.com
✓ Authentication Endpoints > GET /auth/google/callback > should redirect to dashboard for an existing user
✓ Authentication Endpoints > GET /auth/google/callback > should redirect to finish-signup for a new user
✓ Authentication Endpoints > POST /api/register/google-finalize > should return 401 if temp_token is missing or invalid
✓ Authentication Endpoints > POST /api/register/google-finalize > should return 400 if role is invalid
✓ Authentication Endpoints > POST /api/register/google-finalize > should successfully create user with valid role and set session token
error creating user: {
  message: "Database error",
}
✓ Authentication Endpoints > POST /api/register/google-finalize > should return 500 if database error occurs
✓ Authentication Endpoints > GET /api/profile > should return 401 if session token is missing or invalid
Authenticated existing user: profile@test.com
✓ Authentication Endpoints > GET /api/profile > should return profile data with valid session token
✓ Authentication Endpoints > POST /api/logout > should return 200 and clear session token [16.00ms]

 11 pass
 0 fail
 36 expect() calls
Ran 11 tests across 1 file. [225.00ms]
```

File screenshots (snippets only since it's long):

```
dist            backend        auth.test.js
node_modules    frontend       setup.js
public
src
tests
```

```
backend         auth.test.js
frontend        setup.js

import { describe, it, expect, mock } from "bun:test";
import { treaty } from "@elysiajs/eden";
import { app } from "../../src/server";
import { supabase } from "./setup";

// see https://elysiajs.com/patterns/unit-test
// we pass an elysia instance o Eden Treaty to interact with Elysia se
const api = treaty(app);

// 'describe' groups tests
describe("Authentication Endpoints", () => {
  describe("GET /auth/google/url", () => {
    // 'it' creates one test
    it("should return a Google auth URL and set CSRF cookies", async (
      // make the api call (remember we mock arctic in setup.js too)
      const { data, response, error } = await api.auth.google.url.get(

      // check for successful response
      expect(response.status).toBe(200);
      expect(error).toBe(null);
      expect(data).not.toBe(null);

      // check for valid url from data
      expect(typeof data.url).toBe("string");
      expect(data.url).toBe("http://mock.google.auth.url/");

      // check that cookies were set properly
      const cookieHeader = response.headers.get("Set-Cookie") || "";
      expect(cookieHeader).toInclude("google_oauth_state=");
      expect(cookieHeader).toInclude("google_oauth_code_verifier=");
      expect(cookieHeader).toInclude("HttpOnly");
      expect(cookieHeader).toInclude("Path=/");
      expect(cookieHeader).toInclude("SameSite=Lax");
    });
  });

  describe("GET /auth/google/callback", () => {
    it("should return 400 if state is invalid", async () => {
      // simulate a case where state in cookie is different from state
      const storedState = "state_in_cookie";
      const queryState = "state_in_query";
NOR 12.4K  auth.test.js                                          Top  1/2
```

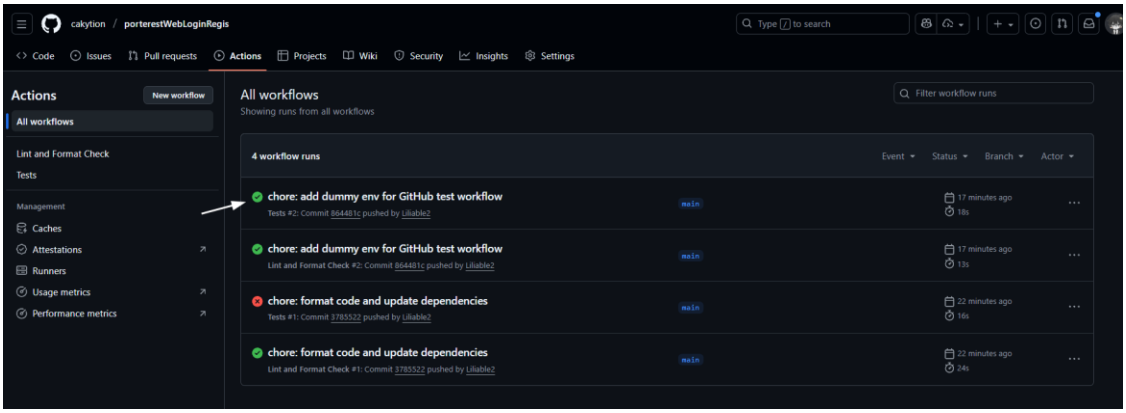
```
dist                    backend                    AuthContext.test.jsx
node_modules            frontend                   FinishSignup.test.jsx
public                                             ProtectedRoute.test.jsx
src                                                setup.js
tests
```

```
backend        AuthContext.test.jsx
frontend       FinishSignup.test.jsx
               ProtectedRoute.test.jsx
               setup.js
```

```
import { render, screen, waitFor } from "@testing-library/react";
import { AuthProvider, useAuth } from "../../src/AuthContext";

// component for testing
function TestComponent() {
  // get auth context
  const { user, isLoading } = useAuth();

  // displays isLoading and user email
  return (
    <div>
      <div data-testid="loading">{isLoading.toString()}</div>
      <div data-testid="user">{user ? user.email : "null"}</div>
    </div>
  );
}

describe("AuthContext", () => {
  // before each test
  beforeEach(() => {
    // replace fetch with a mock and set env for backend url
    global.fetch = vi.fn();
    import.meta.env.VITE_BACKEND_URL = "http://localhost:8080";
  });

  afterEach(() => {
    // clean up mocks after each test
    vi.restoreAllMocks();
  });

  it("should start with isLoading true, then check session", async ()
    // mock fetch to backend to return a successful call with email wi
    global.fetch.mockResolvedValueOnce({
      ok: true,
      json: async () => ({ email: "test@example.com" }),
    });

    // wrap test component with authprovider, render to 'screen'
    render(
      <AuthProvider>
        <TestComponent />
```

```
NOR  2.5K  AuthContext.test.jsx                                  Top  1/4
```

GitHub CI screenshots:

```
D:\university\Year 3\Classes\CPE334 SOFTWARE ENGINEERING\Projects\porterest-repo\.github\workflows
workflows      lint.yml
               test.yml
```

```
name: Tests

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    env:
      NODE_ENV: test
      JWT_SECRET: test-secret-key-for-ci
      SUPABASE_URL: https://test.supabase.co
      SUPABASE_SERVICE_ROLE_KEY: test-service-role-key
      GOOGLE_CLIENT_ID: test-google-client-id
      GOOGLE_CLIENT_SECRET: test-google-client-secret
      BACKEND_URL: http://localhost:8080
      FRONTEND_URL: http://localhost:5173

    steps:
      - uses: actions/checkout@v4

      - uses: oven-sh/setup-bun@v1

      - name: Install dependencies
        run: cd porterest && bun install

      - name: Run tests
        run: cd porterest && bun run test

      - name: Build
        run: cd porterest && bun run build
```